Digital Design (CS F215) **18-11-2024**

Lab Compre : Serial Adder

Lecturer: Harikrishnan N B *Student:*

READ THE FOLLOWING CAREFULLY:

Honour Code for Students: I shall be honest in my efforts and will make my parents proud. Write the oath and sign it on the assignment.

The lab evaluation consists of two parts.

1) **Step 1 (Implementation):** Use iverilog to implement your design. Follow the modeling specified in each question, as only solutions using the instructed modeling will be accepted for evaluation.

2) **Step 2 (Submission):** Once you complete the implementation, create a zip file named `CampusID_LabCompre.zip` containing all relevant files and upload it to `quanta.bits-goa.ac.in (local quanta)`.

Note: The submission format should be strictly followed: All files must be organized as `CampusID_LabCompre.zip > CampusID_LabCompre (folder with same name) > all files`. Submission will be graded using Python script and submission in wrong format will result in direct 0 marks.

The submission deadline is 10:45 AM sharp. Any delay in submission will result in a deduction of 0.5 marks per minute.

`Note: The clock period is 10 seconds (with 50% duty cycle). Initialize the clock to 0 for all the testbenches.`

Note: Students are not allowed to modify the parameters of any Verilog module or testbenches provided in the lab. Modifying these parameters will result in a direct score of 0 for that particular question.

Question 1: Full Adder Module[2 marks]

Design a **Full Adder** module in Verilog. This module will be used in the Serial Adder to perform bitwise addition. The module should be coded using **behavioral modeling** and have the following ports:

- `input a, b, cin`: The input bits for the full adder.
- `output reg sum, cout`: The resulting sum and carry-out.

Question 2: D Flip-Flop Module[1 marks]

Implement a **D Flip-Flop** (DFF) module in Verilog. This module will serve as a storage element in the Serial Adder for bitwise addition. The D Flip-Flop should work on an **asynchronous active-high reset**

and trigger on the **positive edge of the clock**. The module should have:

- `input D, clk, rst`: Data input, clock, and reset signal.
- `output Q`: The stored output of the DFF.

Question 3: Shift Register Module [4 marks]

Design a **Shift Register** module in Verilog using **structural modeling**. This shift register will be used to shift bits serially in the Serial Adder. The shift register should be constructed by instantiating multiple `Dff_shift` modules, where each `Dff_shift` serves as a separate D Flip-Flop with an initial value input to load during reset. Students are required to implement the `Dff_shift` module independently, and this module should be distinct from any other D Flip-Flops used elsewhere.

The `Shiftreg` module should include the following ports:

- `input clk, rst, ctrl, Incoming, num`: Clock, reset, control signal, incoming bit from the left side, and a 4-bit input `num` to initialize the register.
- `output [3:0] out`: 4-bit data output after shifting.

The shift register should operate as follows:

- When `rst` is high, the register loads the initial value provided by `num`, independent of the `ctrl` signal.
- When `rst` is low and `ctrl` is high, the register shifts bits to the right by one position on each clock cycle, with the `Incoming` bit entering from the left (MSB).

Students should implement the `Dff_shift` module with the following specifications:

- `input D, clk, rst, init_val`: The data input, clock, reset, and initial value to load during reset.
- `output Q`: The stored output of the D Flip-Flop.

The `Dff_shift` should have an **active-high, asynchronous reset** and should update its output `Q` on the **positive edge of the clock signal** (`clk`). Ensure that each `Dff_shift` instance is correctly instantiated within the `Shiftreg` module to reflect this functionality.

Question 4: Top Module for Serial Adder [3 marks]

Combine the Full Adder, D Flip-Flop, and Shift Register modules to implement the **Serial Adder**. This module will perform serial addition by shifting bits one by one and adding them, with the carry being stored for the next cycle.

In the `top_module`, ensure the following design constraints:

- Use exactly **2 Full Adders** to handle the serial addition process.
- Use **1 D Flip-Flop** to store and propagate the carry bit between addition cycles.

- Use **2 Shift Registers**:
  - The first Shift Register should store the final cumulative sum of the addition.
  - The second Shift Register should provide the second operand for the addition, with its serial input set to `0`.
- At each clock cycle, the `top_module` should output the current sum result.

This setup will emulate a serial adder's behavior by shifting each bit into the Full Adders, storing the carry in the D Flip-Flop, and accumulating the result in the first Shift Register. Ensure all connections align with the intended Serial Adder operation.